

Before / After: My Own Kickoff Prompt

A comparison of the prompt that started this project versus a rewrite applying what's in `prompting-guide.md` - not to an agent's system prompt this time, but to the request that kicked off the whole build. The same principles that make a good system prompt make a good request to a coding assistant.

Before (the actual first message of this session, verbatim)

I want to build a 3 langraph agnets taht takes user qurrss from cli it acn in multi line text , even .txt aand pdf file shoudl take in put all thisformats 3 agents are oen is summariazarion of health report and second is synthesis health report generation and trod is normal qna bot and i wnat u to bild exactly accroding to my architue palna ia , givimng to use user query must at frist go through input guardlils sinc e its a helatcare rproject it it fAILS IT Should ask useres senstive topic or nay things agter that passes its houdl go to query contextualiters tejre shouls be calslifiacxtion of what tod o sumamrize, synthesis data or nomr;a qna amd tshi shoudl eb done by llm we will use groq avaiable modle fro thsi i ahve its api key. for thsi initialize project in rshi diractory us uv and calude.md fiel fro thsi befroe building 3 aggest i will tell u oen by own wwWhats nedds to be done

It worked - the whole project got built from it - but it took real effort to parse: one run-on paragraph mixing architecture, tech constraints, and process instructions together, with the guardrail's failure behavior and the "build incrementally" instruction both buried mid-sentence.

After (rewritten applying prompt engineering practices)

I want to build a CLI healthcare assistant using LangGraph and Groq-hosted LLMs.

>

*****Input:**** the CLI should accept multi-line pasted text, or a `.txt` or `.pdf` file.*

>

*****Pipeline:*****

- 1. Input Guardrail - an LLM-based safety check. If a query fails (unsafe, off-domain, or a prompt injection attempt), don't just reject it - ask the user to clarify or rephrase around a*

legitimate health topic.

2. *Query Classifier* - an LLM call that routes a passing query to exactly one of three agents: ***summarize*** (user has an existing report), ***synthesize*** (generate a new report by interviewing the user), or ***qna*** (general health question).

3. *Three agents, each its own LangGraph graph.*

>

***Constraints:** use `uv` for the project; use a Groq-hosted model (I have an API key); document the architecture in `CLAUDE.md` before any agent code is written.*

>

***Process:** start with just the project scaffold and `CLAUDE.md`. I'll direct you to build each of the 3 agents one at a time after that, then the guardrail/classifier routing last.*

What changed, and why it matters

Change: One run-on paragraph -> numbered pipeline steps

Principle: Structure mirrors the actual architecture

Why it helps: A linear graph (guardrail -> classifier -> 3 agents) is much easier to build correctly from a numbered list than reverse-engineered from prose

Change: Failure behavior stated as its own bullet ("ask to clarify," not just "if it fails")

Principle: Say the actual rule, not just the category

Why it helps: Same lesson as the classifier fix in `prompting-guide.md` #5 - an implicit rule buried in a sentence is easy to under-specify or miss entirely

Change: Input format, pipeline, constraints, and process split into separate labeled sections

Principle: Separate concerns instead of interleaving them

Why it helps: Each section can be read (and satisfied) independently; nothing about tech stack accidentally reads like part of the architecture

Change: Explicit "start with just the scaffold" process note, stated up front

Principle: Define scope for **this step**, not just the end state

Why it helps: Prevents scope creep - without it, an assistant might reasonably try to build all 3 agents immediately instead of waiting for direction

Change: No typos / unambiguous grammar

Principle: Reduce the reader's need to guess intent

Why it helps: Directly parallels `prompting-guide.md` #5: small/fast models (and humans skimming a request) lean on surface wording - ambiguity gets resolved by guessing, which is exactly where things go wrong

The original prompt's **content** was actually good - the architecture it described is exactly what got built, unchanged. What improved is **legibility**: the same information, organized so the reader (human or model) doesn't have to do parsing work the writer could have done once.